

05 — Repliche, Rolling Updates e Rollback

Modulo: 3 — Deploy di Applicazioni

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Scalare manualmente un Deployment con `oc scale` e verificare il risultato
- Configurare un `HorizontalPodAutoscaler` (HPA) per lo scaling automatico basato su metriche
- Comprendere i parametri `maxSurge` e `maxUnavailable` con esempi numerici concreti
- Scegliere tra strategia `RollingUpdate` e `Recreate` in base al tipo di applicazione
- Eseguire rollback a una revisione specifica con `oc rollout undo`
- Configurare un `PodDisruptionBudget` per garantire disponibilità durante gli aggiornamenti
- Implementare strategie Blue-Green e Canary deployment tramite Route weight su OCP

1. Teoria e Concetti Chiave

Scaling Manuale vs Automatico

Scaling manuale: l'operatore modifica direttamente `spec.replicas` nel Deployment tramite `oc scale`. Semplice, ma richiede intervento umano.

Scaling automatico (HPA): il `HorizontalPodAutoscaler` monitora le metriche (CPU, memoria, metriche custom) e aggiusta automaticamente `spec.replicas` entro limiti configurati (`minReplicas` / `maxReplicas`).

Strategia RollingUpdate: Esempio Numerico

Con `replicas: 4`, `maxSurge: 1`, `maxUnavailable: 0`:

Inizio:	[v1][v1][v1][v1]	(4 Pod v1)
Step 1 - Surge:	[v1][v1][v1][v1][v2]	(1 Pod v2 creato, totale 5)
Step 2 - Down:	[v1][v1][v1] [v2]	(1 Pod v1 eliminato, totale 4)
Step 3 - Surge:	[v1][v1][v1][v2][v2]	(1 Pod v2, totale 5)
Step 4 - Down:	[v1][v1] [v2][v2]	(1 Pod v1 eliminato, totale 4)
...continua...		
Fine:	[v2][v2][v2][v2]	(4 Pod v2)

Risultato: `maxUnavailable: 0` garantisce zero downtime (sempre 4 Pod disponibili). `maxSurge: 1` limita il costo aggiuntivo a 1 Pod extra.

Strategia Recreate: Quando Usarla

La strategia `Recreate` elimina **tutti** i Pod prima di crearne di nuovi. Implica downtime ma è necessaria quando:

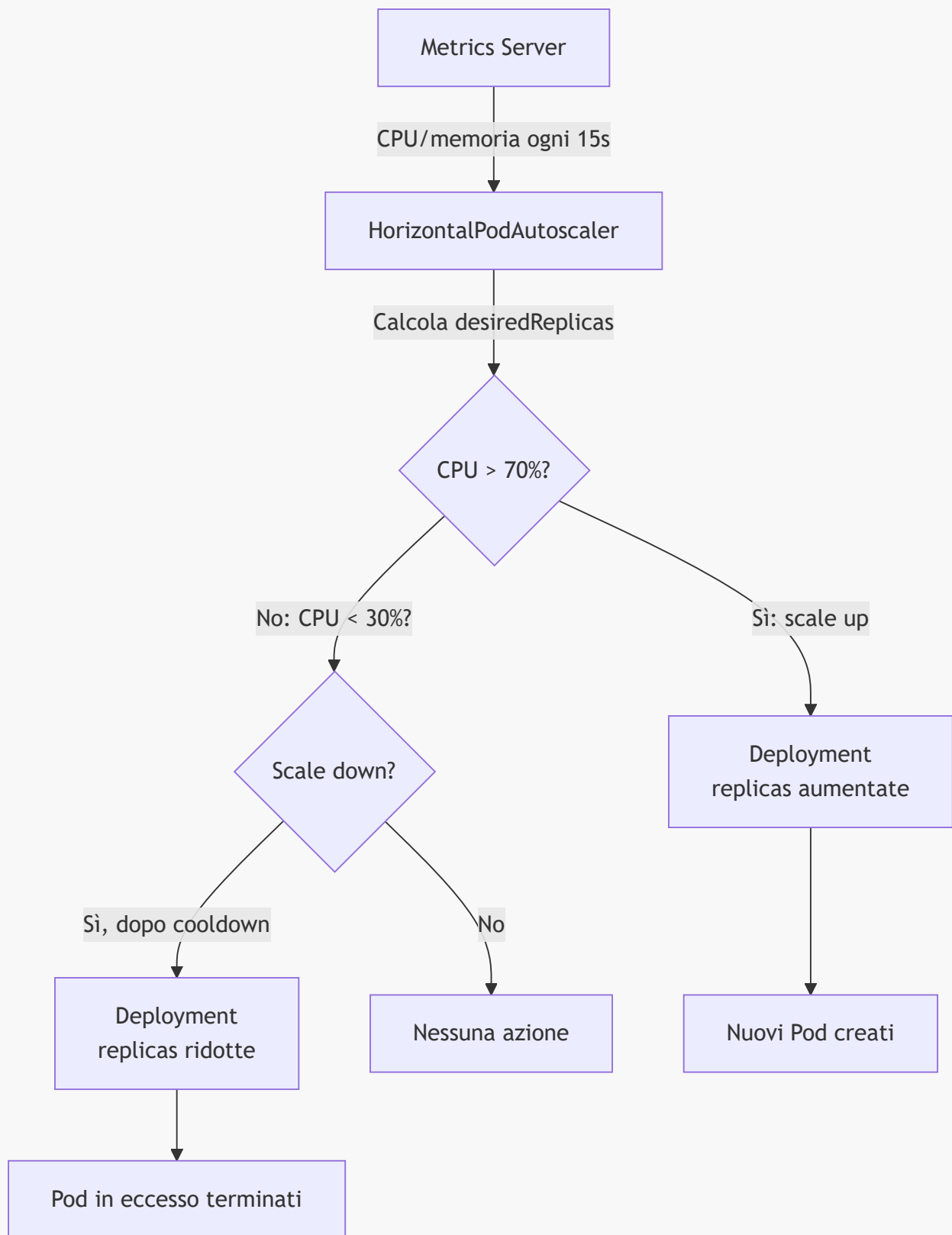
- L'applicazione non supporta più versioni in esecuzione simultanea (es. schema DB con breaking changes)
- La configurazione condivisa (file su storage, lock) non è compatibile con versioni multiple
- Il Pod usa un PVC con **ReadWriteOnce** (che non può essere montato da due Pod contemporaneamente)

Pod Disruption Budget (PDB)

Un **PodDisruptionBudget** limita quanti Pod di un'applicazione possono essere interrotti **volontariamente** contemporaneamente (es. durante un rolling update del nodo, drain, manutenzione). Non si applica a interruzioni involontarie (crash dei nodi).

Senza PDB: durante manutenzione nodo, tutti i Pod su quel nodo vengono rimossi
Con PDB (minAvailable: 2): garantisce che almeno 2 Pod siano sempre disponibili

2. Diagramma — HPA e Scaling Automatico



3. Comandi Pratici con Output Atteso

```
# Scalare manualmente un Deployment a 5 repliche  
oc scale deployment/my-app --replicas=5 -n my-project
```

```

# Output atteso:
# deployment.apps/my-app scaled

# Verificare lo scaling
oc get deployment my-app -n my-project

# Output atteso:
# NAME          READY    UP-TO-DATE    AVAILABLE    AGE
# my-app        5/5      5             5            10m

# Creare un HPA basato su CPU (richiede Metrics Server installato)
oc autoscale deployment/my-app \
  --min=2 \
  --max=10 \
  --cpu-percent=70 \
  -n my-project

# Output atteso:
# horizontalpodautoscaler.autoscaling/my-app autoscaled

# Verificare lo stato dell'HPA
oc get hpa my-app -n my-project

# Output atteso:
# NAME          REFERENCE          TARGETS    MINPODS    MAXPODS    REPLICAS    AGE
# my-app        Deployment/my-app   15%/70%    2          10         3           5m

# Descrizione dettagliata dell'HPA con eventi di scaling
oc describe hpa my-app -n my-project

# --- Rolling Update ---

# Aggiornare l'immagine (avvia un rolling update)
oc set image deployment/my-app \
  my-app=registry.access.redhat.com/ubi9/ubi-minimal:9.3 \
  -n my-project

# Monitorare il progresso del rolling update
oc rollout status deployment/my-app -n my-project

# Output atteso (in tempo reale):
# Waiting for deployment "my-app" rollout to finish: 1 out of 3 new replicas have
# been updated...
# Waiting for deployment "my-app" rollout to finish: 2 out of 3 new replicas have
# been updated...
# Waiting for deployment "my-app" rollout to finish: 2 old replicas are pending
# termination...
# deployment "my-app" successfully rolled out

# Visualizzare la cronologia dei rollout
oc rollout history deployment/my-app -n my-project

# Output atteso:
# REVISION    CHANGE-CAUSE

```

```
# 1      Deploy iniziale v1.0
# 2      Aggiornamento a v1.1
# 3      Aggiornamento a v1.2

# Visualizzare il dettaglio di una revisione
oc rollout history deployment/my-app --revision=2 -n my-project

# Output atteso:
# deployment.apps/my-app with revision #2
# Pod Template:
#   Labels:      app=my-app
#               pod-template-hash=6d8b9c7f5
#   Annotations: kubernetes.io/change-cause: Aggiornamento a v1.1
#   Containers:
#     my-app:
#       Image: registry.access.redhat.com/ubi9/ubi-minimal:9.2
#     ...

# Rollback all'ultima revisione stabile
oc rollout undo deployment/my-app -n my-project
# Output: deployment.apps/my-app rolled back

# Rollback a una revisione specifica
oc rollout undo deployment/my-app --to-revision=1 -n my-project

# Verificare i Pod dopo il rollback
oc get pods -n my-project -l app=my-app

# Output atteso:
# NAME                                READY   STATUS    RESTARTS   AGE
# my-app-5c7d6b8f9-abc12             1/1     Running   0           30s
# my-app-5c7d6b8f9-def34             1/1     Running   0           25s
# my-app-5c7d6b8f9-ghi56             1/1     Running   0           20s
```

4. Esempi YAML Completi

HorizontalPodAutoscaler v2 (con metriche CPU e memoria)

```
# file: hpa-my-app.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app
  namespace: my-project
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
# Deployment da scalare
# minimo 2 Pod (evita scale-to-zero)
```

```

maxReplicas: 10           # massimo 10 Pod
metrics:
# Metrica CPU – scala quando la media supera il 70%
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
# Metrica Memoria – scala quando la media supera 200Mi
- type: Resource
  resource:
    name: memory
    target:
      type: AverageValue
      averageValue: 200Mi
behavior:
  scaleUp:
    stabilizationWindowSeconds: 60    # attende 60s prima di scalare su
    policies:
      - type: Pods
        value: 2                      # scala al massimo 2 Pod per volta
        periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300  # attende 5 min prima di scalare giù
      policies:
        - type: Pods
          value: 1                    # riduce al massimo 1 Pod per volta
          periodSeconds: 120

```

Deployment con strategia RollingUpdate configurata

```

# file: deployment-rolling.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: my-project
  annotations:
    kubernetes.io/change-cause: "Deploy versione 1.2.0"
spec:
  replicas: 4
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                # mai più di 5 Pod totali durante l'aggiornamento

```

```

    maxUnavailable: 0      # zero downtime: tutti i 4 Pod disponibili durante
l'aggiornamento
    minReadySeconds: 15    # attende 15s che il Pod sia stabile prima di
procedere
    progressDeadlineSeconds: 300 # fallisce se il rollout non completa entro 5 min
    revisionHistoryLimit: 5
    template:
      metadata:
        labels:
          app: my-app
      spec:
        containers:
        - name: my-app
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.3
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
            successThreshold: 1
            failureThreshold: 3

```

Deployment con strategia Recreate

```

# file: deployment-recreate.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-stateful-app
  namespace: my-project
spec:
  replicas: 1      # spesso usato con replicas: 1 per app con storage
esclusivo
  selector:
    matchLabels:
      app: my-stateful-app
  strategy:
    type: Recreate # elimina tutti i Pod prima di crearne di nuovi
  template:
    metadata:
      labels:
        app: my-stateful-app
    spec:

```

```

containers:
- name: my-stateful-app
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.3
  volumeMounts:
  - name: data
    mountPath: /data
volumes:
- name: data
  persistentVolumeClaim:
    claimName: my-app-pvc # PVC ReadWriteOnce: non montabile da 2 Pod
                           contemporaneamente

```

PodDisruptionBudget

```

# file: pdb-my-app.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pdb-my-app
  namespace: my-project
spec:
  # Opzione 1: minAvailable – almeno N Pod devono sempre essere disponibili
  minAvailable: 2 # con 4 repliche: al massimo 2 Pod possono essere
interrotti
  selector:
    matchLabels:
      app: my-app

# Opzione alternativa: maxUnavailable – al massimo N Pod possono essere non
disponibili
# spec:
#   maxUnavailable: 1
#   selector:
#     matchLabels:
#       app: my-app

```

Blue-Green Deployment con Route weight

```

# file: route-bluegreen.yaml
# Blue-Green: due Deployment (blue e green), la Route punta a uno solo alla volta
# Per il deploy: si crea il Deployment green, si verifica, poi si sposta il
traffico

# Step 1: Route punta al Deployment blue (100%)
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: my-app
  namespace: my-project

```



```
spec:
  to:
    kind: Service
    name: my-app-blue      # traffico al Service blue
    weight: 100
  alternateBackends:
  - kind: Service
    name: my-app-green    # green pronto ma traffico 0%
    weight: 0
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

```
# Blue-Green: spostare il traffico da blue a green
oc patch route my-app -n my-project \
  -p '{"spec":{"to":{"name":"my-app-green","weight":100},"alternateBackends":
[{"name":"my-app-blue","weight":0}]}}'
```

```
# Canary: inviare il 20% del traffico alla nuova versione (green)
oc patch route my-app -n my-project \
  -p '{"spec":{"to":{"name":"my-app-blue","weight":80},"alternateBackends":
[{"name":"my-app-green","weight":20}]}}'
```

```
# Verificare la distribuzione del traffico
oc get route my-app -n my-project -o yaml | grep -A5 "alternateBackends"
```

5. Note Specifiche per Azure ARO

```
# Su ARO, il Metrics Server è pre-installato: l'HPA funziona out-of-the-box
oc get deployment metrics-server -n openshift-monitoring
# Output: metrics-server è gestito da Red Hat/Microsoft come parte del cluster
```

```
# Verificare che le metriche siano disponibili per l'HPA
oc top pods -n my-project
```

```
# Output atteso:
```

# NAME	CPU(cores)	MEMORY(bytes)
# my-app-5c7d6b8f9-abc12	5m	45Mi
# my-app-5c7d6b8f9-def34	8m	48Mi

```
# Su ARO, durante il node drain (aggiornamento cluster), il PDB viene rispettato:
# se minAvailable non è soddisfatto, il drain si blocca fino a che i Pod vengono
# riprogrammati su altri nodi
oc get pdb -n my-project
```

```
# Output atteso:
```

#	NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS	AGE
#	pdb-my-app	2	N/A	2	1h

⚠ **Attenzione ARO:** Durante gli **aggiornamenti automatici del cluster ARO** (gestiti da Microsoft/Red Hat), i nodi vengono aggiornati con un rolling drain. Configurare sempre un PDB per le applicazioni critiche per evitare che tutti i Pod vengano eliminati contemporaneamente durante la manutenzione dei nodi.

💡 **Suggerimento ARO — Blue-Green:** Su ARO, le Route usano il wildcard DNS `*.apps.<cluster-name>.<region>.aroapp.io`. Entrambi i Service blue e green devono essere creati prima di creare la Route con `alternateBackends`.

Aspetto	OCP On-Premise	Azure ARO
Metrics Server	Da installare manualmente	Pre-installato e gestito
HPA	Disponibile dopo Metrics Server	Disponibile out-of-the-box
PDB durante node drain	Rispettato da <code>oc adm drain</code>	Rispettato durante aggiornamenti ARO automatici
Blue-Green DNS	Configurato dall'admin	Wildcard ARO automatico
Custom Metrics	Prometheus Adapter	Azure Monitor + Prometheus Adapter

6. Riepilogo dei Punti Chiave

- `oc scale deployment/<nome> --replicas=N` scala manualmente; l'**HPA** scala automaticamente in base a metriche CPU/memoria
- `maxSurge` controlla quanti Pod extra si creano durante il rolling update; `maxUnavailable` controlla quanti possono essere non disponibili
- Con `maxSurge: 1` e `maxUnavailable: 0` si ottiene **zero downtime** garantito durante il rolling update
- `Recreate` elimina tutti i Pod prima di crearne di nuovi: necessario per app con PVC `ReadWriteOnce` o schema DB non retrocompatibile
- `oc rollout undo --to-revision=N` esegue il rollback a qualsiasi revisione nella cronologia (fino a `revisionHistoryLimit`)
- Il **PodDisruptionBudget** è essenziale su ARO per garantire disponibilità durante gli aggiornamenti automatici dei nodi del cluster
- **Blue-Green** e **Canary** su OCP si implementano tramite `Route` con `alternateBackends` e `weight`, senza tool aggiuntivi

7. Domande di Verifica

1. Con `replicas: 4`, `maxSurge: 1`, `maxUnavailable: 0`, quanti Pod sono in esecuzione al massimo durante un rolling update?

- A) 4
- B) 5 ✓

- C) 6
- D) 3

2. Quale strategia di deployment è obbligatoria quando l'applicazione usa un PVC con `ReadWriteOnce`?

- A) RollingUpdate con `maxUnavailable: 1`
- B) RollingUpdate con `maxSurge: 0`
- C) **Recreate** ✓
- D) Blue-Green

3. Cosa garantisce un `PodDisruptionBudget` con `minAvailable: 2`?

- A) Che almeno 2 Pod siano in fase di aggiornamento
- B) **Che almeno 2 Pod siano sempre disponibili durante interruzioni volontarie (drain, manutenzione)** ✓
- C) Che il Deployment abbia sempre almeno 2 repliche
- D) Che l'HPA non scali sotto le 2 repliche

4. Come si implementa un Canary deployment al 20% su OpenShift?

- A) Configurando `canaryReplicas: 1` nel Deployment
- B) Usando un HPA con soglia del 20%
- C) **Modificando il `weight` dei `alternateBackends` nella Route (es. main: 80, canary: 20)** ✓
- D) Creando due namespace separati e bilanciando con un LoadBalancer esterno

5. Su ARO, perché è particolarmente importante configurare un `PodDisruptionBudget`?

- A) Perché ARO non supporta rolling update nativi
- B) Perché l'HPA di ARO ignora le repliche minime
- C) **Perché ARO esegue aggiornamenti automatici dei nodi che drenano i Pod; senza PDB tutti i Pod potrebbero essere rimossi contemporaneamente** ✓
- D) Perché ARO limita il numero massimo di repliche a 5